JAVA DATE & TIME

Temporal Programming with java.time

codehive

Essential Classes

Java managees dates and times through the **java.time** API (introduced in Java 8). It is more robust and predictable than the older Calendar classes.

Class	Representation
LocalDate	A date (year, month, day) e.g., 2026-02-01
LocalTime	Time (hour, minute, second, nanoseconds)
LocalDateTime	Combined Date and Time instance
DateTimeFormatter	The engine for custom formatting and parsing

1. Working with LocalDate

The `LocalDate` class represents an ISO-8601 calendar system date without a time zone.

```java
import java.time.LocalDate;

public class Main {
  public static void main(String[] args) {
    LocalDate today = LocalDate.now(); // Static factory method
    System.out.println(today);
  }
}
```

CONSOLE OUTPUT

```
2026-02-01
```

2. Capturing LocalTime

Use `LocalTime` when you only care about the wall-clock time (e.g., Opening hours or countdowns).

```java
import java.time.LocalTime;

public class Main {
  public static void main(String[] args) {
    LocalTime currentTime = LocalTime.now();
    System.out.println(currentTime);
  }
}
```

CONSOLE OUTPUT

```
19:01:27.753421
```

3. Combined LocalDateTime

This class is useful for timestamps where both the specific day and specific time are required.

```
import java.time.LocalDateTime;

public class Main {
  public static void main(String[] args) {
    LocalDateTime dt = LocalDateTime.now();
    System.out.println(dt); // Separated by a 'T'
  }
}
```

Note: The standard ISO format uses a "T" to separate the date from the time (e.g., 2026-02-01T19:00:00).

4. Formatting Date and Time

Raw ISO strings are rarely user-friendly. `DateTimeFormatter` allows you to transform these objects into readable strings.

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

LocalDateTime myDateObj = LocalDateTime.now();
DateTimeFormatter pattern = DateTimeFormatter.ofPattern("dd-MM-yyyy
HH:mm:ss");

String formatted = myDateObj.format(pattern);
System.out.println(formatted);
```

Pattern Reference Table

Pattern	Resulting Example
yyyy-MM-dd	2026-02-01
dd/MM/yyyy	01/02/2026
dd-MMM-yyyy	01-Feb-2026
E, MMM dd yyyy	Sun, Feb 01 2026

Summary & Best Practices

- ✔ Always prefer **java.time** over the legacy java.util.Date.
- ✔ Objects in this package are **immutable** and thread-safe.
- ✔ Use `.now()` for current system time.
- ✔ Use `DateTimeFormatter` for human-readable exports.

Ready to build professional Java applications?

Master these date classes to handle logs, schedules, and user timestamps effectively.

Produced by

codehive

Knowledge Accelerated.